# What is Python?

A language designed to be read almost like plain English

## The simple definition

Python is a general-purpose programming language known for its simple, readable syntax that resembles plain English.

### How it compares:

| | |
|---|---|
| Java | `System.out.println("Hello");` |
| C++ | `std::cout << "Hello" << endl;` |
| Python | `print("Hello")` |

## Where Python is used



### Web Development

Backend & APIs (Django, FastAPI)



### AI & Machine Learning

TensorFlow, PyTorch, scikit-learn



### Data Analysis & Visualization

pandas, matplotlib, seaborn



### Automation & Scripting

File tasks, scheduling, ETL



### Web Scraping

BeautifulSoup, Scrapy



### Game Development

Pygame and more

# Why Python for Data Analytics?

What you can do with Python that you can't easily do elsewhere

## 🧹 Advanced Data Cleaning

Handle mixed date formats, messy text, nulls, and duplicates across 50,000+ rows in seconds. Power Query can't match the flexibility.

## 📋 Report Automation

Write a script once — it reads your data, cleans it, calculates KPIs, and emails the report automatically. Zero manual steps every Monday morning.

## 🕸️ Web Scraping

Pull live prices, job postings, or competitor data straight from websites into a DataFrame — no copy-pasting from a browser.

## 🔍 Exploratory Data Analysis

groupby, describe(), pivot_table, correlation matrices — answer business questions in 3 lines of code instead of 10 clicks in Excel.

## 🔌 APIs

Connect directly to Google Sheets, Salesforce, or any data source via its API. Automate the data pull so your dashboard is always fresh.

# Python vs Excel & SQL — Why Add Python?

You already know powerful tools — here's what Python adds on top

## Excel / Power Query

You know this ✓

### ✅ Strengths

· Quick visual data entry

· Point-and-click pivot tables

· Great for small datasets

· Power Query for basic ETL

· Charts in a few clicks

### ⚠️ Limitations

· Slows down past ~100k rows

· Manual steps = hard to repeat

· No version control

· Limited text/date manipulation

· Can't connect to APIs

## SQL

You know this ✓

### ✅ Strengths

· Query millions of rows fast

· Joins across tables

· Aggregations & grouping

· Standard across all databases

· Great for data extraction

### ⚠️ Limitations

· Can't easily clean messy text

· No machine learning

· No visualisation

· One query at a time

· Hard to automate workflows

## Python + pandas

You're learning this 🐼

### ✅ Strengths

· Handles 10M+ rows easily

· Full automation — runs unattended

· Reads Excel, CSV, SQL, APIs

· Cleans, analyses, visualises

· Scales to ML & AI when ready

### ⚠️ Limitations

· Steeper learning curve

· No built-in GUI

· Need to install libraries

· Syntax errors can be frustrating

· **(All solvable with practice 💪)**

# The Data Analyst's Python Stack

Four libraries — that's all you need to get started

## 🐼 pandas

Your Excel replacement

DataFrames, data cleaning, groupby, merge, pivot tables. If you only learn one library — make it this one.

```
import pandas as pd
```

## 🔢 NumPy

The maths engine

Fast number crunching, arrays, statistical functions. pandas is built on top of it — you'll use it for np.where() and np.nan.

```
import numpy as np
```

## 📈 matplotlib

Your chart engine

Bar charts, line charts, histograms, scatter plots. Not the prettiest by default but completely customisable.

```
import matplotlib.pyplot as plt
```

## 🎨 seaborn

Beautiful stats charts

Built on matplotlib — adds boxplots, heatmaps, distribution plots with one line of code. Great for exploring data.

```
import seaborn as sns
```

# Python for Data Analytics

Session 1 — Python Foundations

Variables & Types | Lists | Dictionaries | if / for | pandas Intro

pandas · Google Colab · Jupyter Notebook

# Session 1 — What We'll Cover

## 01
### Variables & Data Types
str · int · float · bool — and why types matter

## 02
### Lists
Ordered collections, indexing, add/remove, loops

## 03
### Dictionaries
Key-value pairs — the backbone of DataFrames

## 04
### if / elif / else
Decision logic — making your code choose

## 05
### for Loops
Repeating actions over a collection

## 06
### pandas & DataFrames
Series, DataFrames, reading Excel/CSV files

# Your Coding Environment

Before we write a single line of code — let's set up where we write it.

**Google Colab** **Jupyter Notebook**

# Google Colab vs Jupyter Notebook

Same idea, different home — pick the one that fits your setup

## ☁️ Google Colab

`colab.research.google.com`

- ✅ Zero setup — works in any browser
- ✅ Free GPU & TPU for heavy workloads
- ✅ Saves to Google Drive automatically
- ✅ Share like a Google Doc — one link
- ✅ All libraries pre-installed (pandas, numpy…)
- ⚠️ Needs internet connection
- ⚠️ Session times out after ~90 min idle
- ⚠️ Can't access local files easily

👆 **Best for: beginners, no install, sharing work with others**

VS

## 🪐 Jupyter Notebook

`localhost:8888` (runs on your machine)

- ✅ Works fully offline — no internet needed
- ✅ Full access to your local files & folders
- ✅ No session timeout
- ✅ Better for production / scheduled scripts
- ✅ Extensions: Table of Contents, dark mode…
- ⚠️ Requires installation (Anaconda / pip)
- ⚠️ Sharing requires extra steps (nbviewer, git)
- ⚠️ You manage your own library versions

👆 **Best for: local work, large files, offline use, professional env.**

# What is a Notebook?

A document where text, code, and output all live together

💡 Think of it like a Word document — but every paragraph can be

## CELL TYPES:

📝 **Markdown Cell (text / explanation)**

```
## Data Cleaning
This section handles **missing values** and *duplicates*.
Below we'll fill nulls with the column median.
```

💻 **Code Cell (Python that runs)**

In [3]:

```
df['Salary'].fillna(df['Salary'].median())
```

Out [3]:

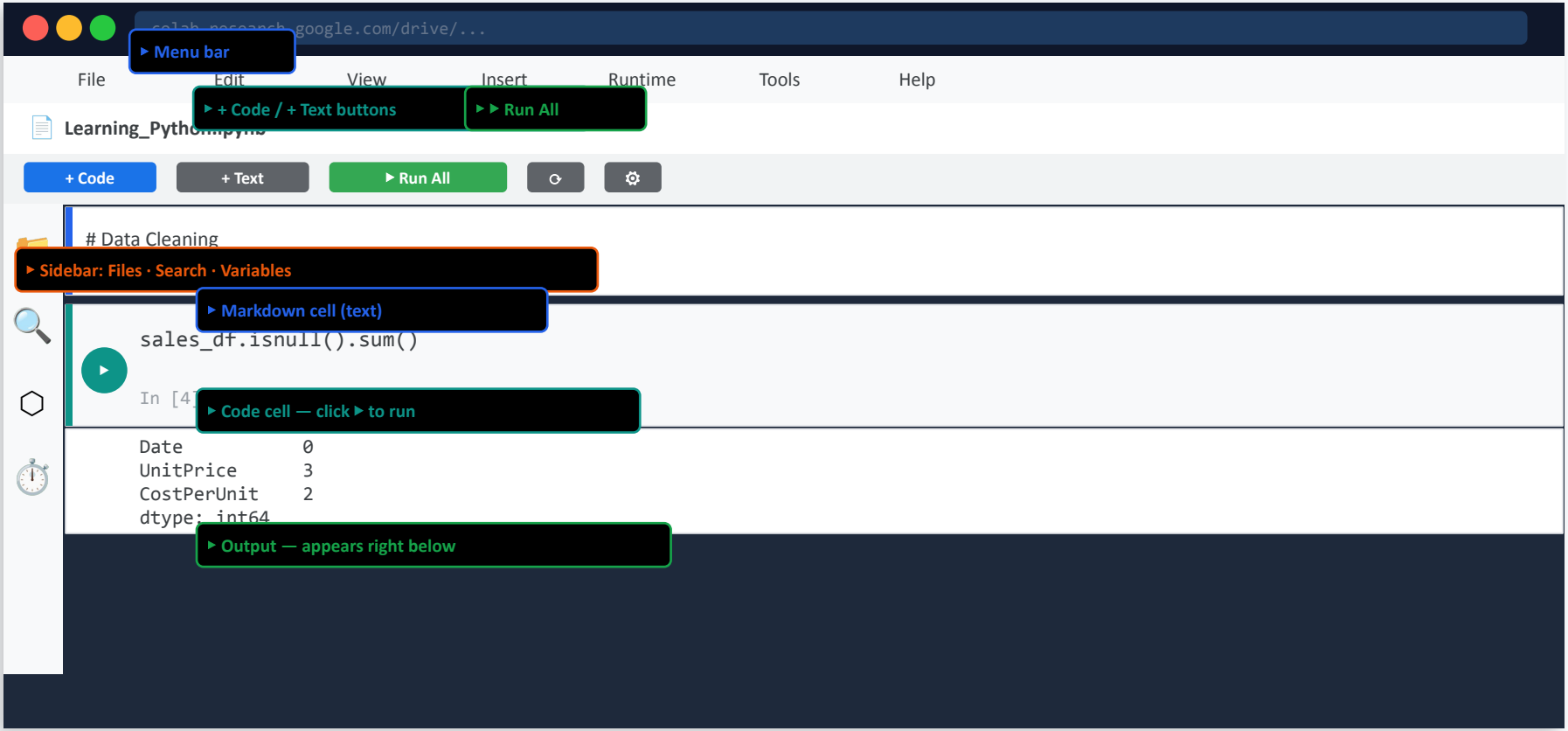
```
0    15000
1    20000    dtype: int64
```

## ⌨ Essential Keyboard Shortcuts

| | |
|---|---|
| `Shift + Enter` | Run cell, move to next |
| `Ctrl + Enter` | Run cell, stay on it |
| `Alt + Enter` | Run cell, insert new below |
| `A` | Insert cell Above (command mode) |
| `B` | Insert cell Below (command mode) |
| `D D` | Delete cell (press D twice) |
| `M` | Change cell to Markdown |
| `Y` | Change cell to Code |
| `Ctrl + /` | Comment/uncomment line |
| `Ctrl + Z` | Undo |
| `Esc` | Enter command mode |
| `Enter` | Enter edit mode |

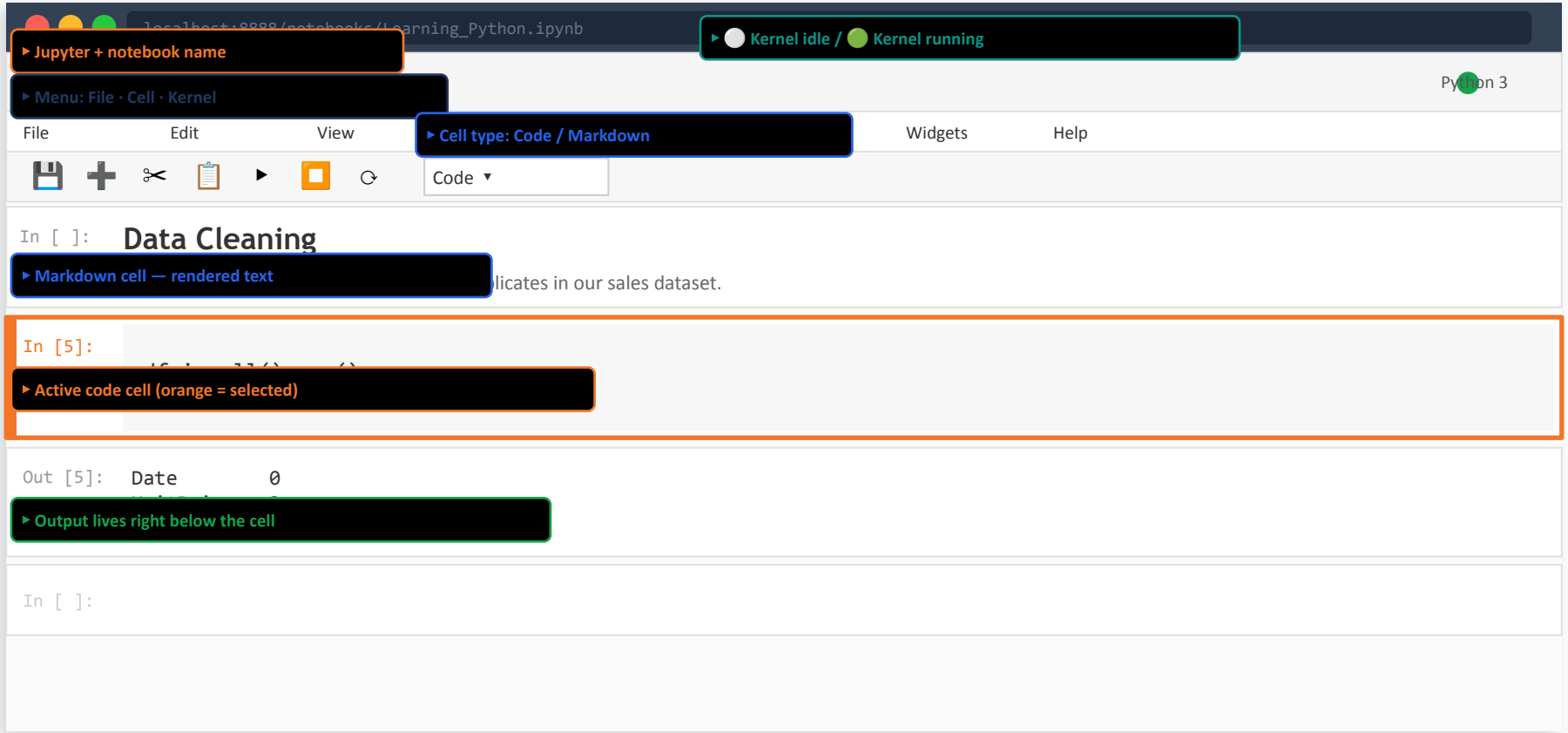
# Google Colab — UI Tour

What every button and panel does



# Jupyter Notebook — UI Tour

Runs locally in your browser — everything on your machine



# Running Your Notebook

The most important actions you'll use every single session

## ▶ Run a Single Cell

```
Shift + Enter
```

```
Ctrl + Enter
```

Run just the selected cell. Output appears immediately below. Use this while building — run one cell at a time to catch errors early.

## ⏭ Run All Cells

```
Runtime → Run All
```

```
(Colab)  or  Cell → Run All
```

Runs every cell from top to bottom in order. Use this when you open a notebook after a break — it re-builds all your variables.

## 🔄 Restart Kernel

```
Runtime → Restart
```

```
(Colab)  or  Kernel → Restart
```

Clears all variables from memory — like opening a fresh Python session. Run this when your code behaves unexpectedly.

### 🐛 Two things that trip everyone up — and how to fix them:

❌ **`NameError: name 'df' is not defined`**
✅ You haven't run the import/read cell yet. Run all cells from the top (Shift+Enter through each).

❌ **`Colab session disconnected / variables lost`**
✅ Runtime → Run All to rebuild your workspace. This is normal — Colab sessions time out.

# Your First Notebook — Step by Step

From zero to running your first line of Python in 2 minutes

## ☁️ Google Colab

1. **Open your browser**
   Go to  colab.research.google.com
2. **Sign in with Google**
   Use any Google / Gmail account
3. **New notebook**
   File → New notebook  (or click + New)
4. **Rename it**
   Click 'Untitled0' → type your name
5. **Type your first cell**
   Click a cell, type  print('Hello!')
   then press  Shift + Enter
6. **See the output**
   'Hello!' appears directly below — you're running Python! 🎉

## 🪐 Jupyter Notebook

1. **Install Anaconda**
   anaconda.com → Download → Install
2. **Open Anaconda Navigator**
   Search 'Anaconda' in Start / Spotlight
3. **Launch Jupyter Notebook**
   Click Launch — browser opens automatically
4. **Navigate to your folder**
   Use the file browser to find your work folder
5. **New notebook**
   New → Python 3 (ipykernel)
6. **Run your first cell**
   Type  print('Hello!')  → Shift + Enter → done! 🎉

# Variables & Data Types

How Python stores and labels information

# Variables

A variable is a labelled box that holds a value

**str**

```
employee_name = "Sara Ahmed"
```

**int**

```
salary = 12000
```

**float**

```
tax_rate = 0.15
```

**bool**

```
is_active = True
```

variables.py

```python
# Creating variables
city = "Cairo"
population = 21_000_000
avg_temp = 29.5
is_capital = True

# Print them
print(city)
print(f"{city} pop: {population}")

# Check the type
print(type(salary))   # <class 'int'>
print(type(avg_temp))  # <float>
print(type(is_capital))# <bool>
```

**str** `"Cairo"` / `"Analyst"`
Text — always in quotes

**int** `42` / `-7` / `0`
Whole numbers

**float** `3.14` / `99.9`
Decimals

**bool** `True` / `False`
Two values only

# Lists

An ordered, changeable collection of items

# Lists

Like a column in Excel — ordered, numbered from 0

| index [0] | index [1] | index [2] | index [3] | index [4] |
|---|---|---|---|---|
| Data Analyst | Sr. Analyst | Data Scientist | Data Engineer | ML Engineer |
| -5 | -4 | -3 | -2 | -1 |

← negative indexing counts from the end

**append(x)**
Add x to the end

```
jobs.append('BI Analyst')
```

**insert(i, x)**
Add x at position i

```
jobs.insert(2, 'Analyst')
```

**extend(list2)**
Merge another list in

```
jobs.extend(other_list)
```

**remove(x)**
Delete first match of x

```
jobs.remove('ML Engineer')
```

**pop()**
Remove & return last item

```
last = jobs.pop()
```

**len(list)**
Count items

```
print(len(jobs))
```

# for Loops & if with Lists

Iterate over every item — like Excel's 'apply to each row'

Imagine this list of scores:




for_if_example.py

```
scores = [50, 70, 100, 120, 80]

# Loop over every score
for score in scores:
    if score >= 100:
        print(f"{score} → A")
    elif score >= 80:
        print(f"{score} → B")
    elif score >= 60:
        print(f"{score} → C")
    else:
        print(f"{score} → F")

# Output:
# 50 → F
# 70 → C
# 100 → A
# 120 → A
# 80 → B
```

💡 The for loop runs the if block once per item.
You can't compare a whole list to a number — loop first!

# Dictionaries

Key-value pairs — like a lookup table or a form

# Dictionaries

Each value is stored under a named key — not a number

## List

| | |
|---|---|
| [0] | Sara Ahmed |
| [1] | 29 |
| [2] | Data Analyst |
| [3] | Cairo |

❓ *What is index 2?*

You'd have to remember what each position means.

## Dictionary

| | |
|---|---|
| "name" | :"Sara Ahmed" |
| "age" | :"29" |
| "job" | :"Data Analyst" |
| "city" | :"Cairo" |

✅ `employee["job"]` → `"Data Analyst"`

Keys are meaningful labels — no memorising positions.

```
employee = {"name": "Sara", "age": 29, "job": "Data Analyst", "city": "Cairo"}
print(employee["job"])          # Data Analyst
print(employee.get("salary", 0)) # 0 (default if key missing)
```

# Dictionary Operations

Access · Change · Add · Remove · Loop

## 📖 Access

```
employee["name"]          # → "Sara"  (KeyError if missing)
employee.get("salary")    # → None  (safe, no error)
employee.keys()           # → all keys
employee.values()         # → all values
employee.items()          # → (key, value) pairs
```

## ✏️ Change & Add

```
employee["age"] = 30                  # update existing key
employee["salary"] = 12000            # add new key
employee.update({"city":"Alex"})      # merge dict in
```

## 🗑️ Remove

```
employee.pop("age")          # remove key, returns value
employee.popitem()           # remove last inserted pair
del employee["salary"]       # delete key (no return)
```

## 🔄 Loop

```
for k in employee:                # loop over keys
for v in employee.values():       # loop over values
for k,v in employee.items():      # loop over both
```

# Dictionary → DataFrame

This is exactly how you build a pandas DataFrame

## Step 1 — Dictionary

```
data = {
  "Name":   ["Sara","Omar","Dina"],
  "Salary": [15000, 20000, 18000],
  "Level":  ["Junior","Senior","Mid"]
}
```

`pd.DataFrame(data)`

## Step 2 — DataFrame

| | Name | Salary | Level |
|---|---|---|---|
| 0 | Sara | 15,000 | Junior |
| 1 | Omar | 20,000 | Senior |
| 2 | Dina | 18,000 | Mid |

🔑 The dictionary KEYS become column names.
The dictionary VALUES (lists) become the column data.
Every list must be the same length — one item per row.

# if / elif / else

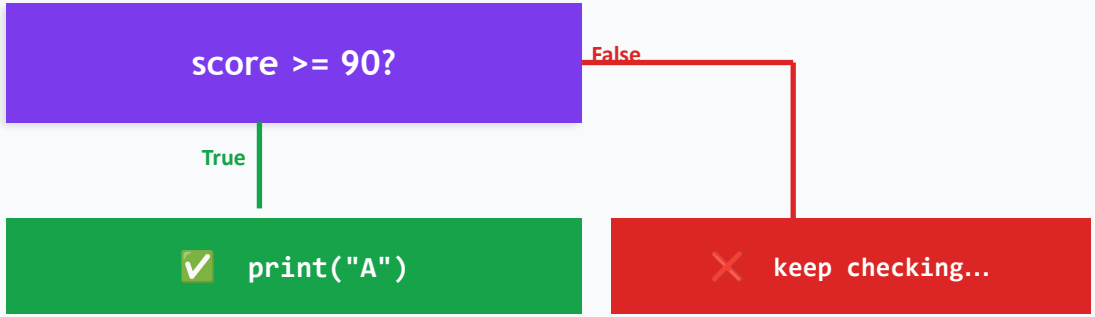
Making your code choose a path based on a condition

# if / elif / else

Decision logic — like Excel's =IFS() but readable

## Comparison operators:

| | |
|---|---|
| `>` greater than | `>=` greater or equal |
| `<` less than | `<=` less or equal |
| `==` equal to | `!=` not equal |




grade_checker.py

```
score = int(input('Enter score: '))

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 60:
    print("C")
else:
    print("F — study harder!")
```

## Combining conditions

```
score >= 60 and score < 80          # both must be true → "and"

score < 60 or score > 100           # either can be true → "or"

not is_active                       # flip True/False  → "not"
```

# for Loops

Run the same block of code for every item in a collection

# for Loops

Like 'Do this for every row' in Excel Power Query

loop_anatomy.py

```
for  employee  in  team_list :
    print(employee['name'])
```

keyword — loop variable (you name it) — keyword — collection to loop over

## Loop over a list

```
cities = ['Cairo','Alex','Giza']
for city in cities:
    print(f"Hello {city}")
```

## Loop with range()

```
# Count 0 to 4
for i in range(5):
    print(i)

# Count 1 to 10
for i in range(1, 11):
    print(i)
```

## Loop a dictionary

```
emp = {'name':'Sara',
       'salary':12000}

for key, value in emp.items():
    print(key, '→', value)

# name → Sara
# salary → 12000
```

# pandas & DataFrames

The library that makes Python feel like a supercharged spreadsheet

# pandas: Series & DataFrame

Two building blocks — column vs table

## Series — a single column

| | |
|---|---|
| 0 | 15,000 |
| 1 | 20,000 |
| 2 | 18,000 |
| 3 | 30,000 |

Name: Salary

```
s = pd.Series([15000,20000,18000,30000])
# One column, auto-numbered index
```

## DataFrame — a full table (multiple Series)

| | Name | Salary | Level |
|---|---|---|---|
| 0 | Sara | 15,000 | Junior |
| 1 | Omar | 20,000 | Senior |
| 2 | Dina | 18,000 | Mid |
| 3 | Khaled | 30,000 | Senior |

```
df = pd.DataFrame(data_dict)
# Each key = column,  each list = column's data
```

# Reading & Exploring Data

The first 5 commands you run on any new dataset

## 📥 Reading Files

```
import pandas as pd

df = pd.read_excel('sales.xlsx')
df = pd.read_csv('sales.csv')
```

**`df.head(5)`**
First 5 rows — quick sanity check

**`df.tail(5)`**
Last 5 rows

**`df.shape`**
(rows, columns) — how big is it?

**`df.dtypes`**
Data type of each column

**`df.info()`**
Dtypes + non-null count in one shot

**`df.describe()`**
Count, mean, std, min, max, quartiles

**`df.isnull().sum()`**
How many nulls per column

**`df.columns`**
List all column names

## 📋 Every time you open a new dataset, run these in order:

1. `df.shape`
2. `df.head()`
3. `df.dtypes`
4. `df.describe()`
5. `df.isnull().sum()`

# List vs Dictionary vs DataFrame

Which one do you use when?

## List [ ]

Ordered collection of values

Access by position: list[0]

Can mix types: [1, 'a', True]

Use when: simple sequence of values

**→ e.g. a column of numbers**

```
employees = ['Sara','Omar','Dina']
```

## Dictionary { }

Key → Value pairs

`Access by name: dict['key']`

Keys must be unique

Use when: labelled record (one row)

**→ e.g. one employee's details**

```
emp = {'name':'Sara','salary':12000}
```

## DataFrame

Table with rows and named columns

`Access by name: df['column']`

Built from a dict of equal-length lists

Use when: multi-row tabular data

**→ e.g. your whole Excel sheet**

```
df = pd.DataFrame(dict_of_lists)
```

List ——builds——▶ Dictionary ——feeds——▶ DataFrame ——powers——▶ Analysis

# Session 1 — What You Now Know

### 📦 Variables & Types

str · int · float · bool
Store and label any value

### 📋 Lists

Ordered, indexed collection
add/remove/loop

### 🗂️ Dictionaries

Key → Value pairs
the DNA of a DataFrame row

### 🔀 if / elif / else

Decision logic
== · >= · and · or · not

### 🔄 for Loops

Iterate over lists & dicts
range() · .items()

### 🐼 pandas DataFrame

Read Excel/CSV, explore shape
head() · info() · describe()

Next: Data Cleaning → Outlier Detection → Feature Engineering →